Latency-Sensitive Service Chaining with Isolation Constraints

Yannick Carlinet
Nancy Perrot
Laurent Valeyre
Jean-Philippe Wary
firstname.lastname@orange.com
Orange, Innovation Division
Paris area, France

Krzysztof Bocianiak
Wojciech Niewolski
Aleksandra Podlasek
firstname.lastname@orange.com
Orange Polska, Innovation Division
Warsaw, Poland

ABSTRACT

Multi-access Edge Computing (MEC) is one of the key enablers in 6G. By offering computing capabilities near the end users, it enables lower environmental impact, higher end-to-end latency, higher resiliency, among others. It is also a critical part in the Cloud-Edge-IoT continuum. However, the deployment of services in the edge poses some security challenges. This work aims at solving orchestration issues, such as how to deal with service deployment while meeting service-specific requirements, while at the same time meeting isolation requirements, in a multi-cluster environment. Our algorithm is estimated to be around 20% better than the baseline, while at the same time offering guarantees on the communication latency and isolation requirements, which the baseline cannot offer. We were able to build an orchestration engine, with an advanced algorithm to compute optimal placement of microservices, integrated in a state-of-art emulation platform, with industry best practice. This demonstrates the feasibility in practice and efficiency of our approach.

CCS CONCEPTS

• **Networks** → **Cloud computing**; **Network algorithms**; • **Computing methodologies** → **Model development and analysis**.

KEYWORDS

Edge Computing, Multi-Edge Computing, Latency-sensitive Service, Service Chaining, Security Isolation

ACM Reference Format:
Yannick Carlinet, Nancy Perrot, Laurent Valeyre, Jean-Philippe Wary, Krzysztof Bocianiak, Wojciech Niewolski, and Aleksandra Podlasek. 2024. Latency-Sensitive Service Chaining with Isolation Constraints. In *Proceedings of 1st International Workshop on MetaOS for the Cloud-Edge-IoT Continuum (MECC '24).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

MECC '24, April 22–25, 2024, Athens, Greece

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Multi-access edge computing (MEC) is a distributed computing architecture that brings computing and storage resources closer to the network edge. It enables applications and services to be hosted and processed at the network edge, rather than in centralized data centers or cloud platforms. By bringing computing closer to the end-users, MEC can reduce network latency, improve application performance, and support emerging use cases that require low latency and high bandwidth, such as augmented reality, virtual reality, or autonomous vehicles. MEC can also enable new business models and revenue streams for network operators and service providers, by providing value-added services, such as content caching, video optimization, or network slicing.

The advantages of edge computing include the following.

Reduced latency: Edge computing reduces the time it takes for data to be processed, analyzed and acted upon, as the data is processed locally.

Improved reliability: Edge computing reduces the dependence on a centralized system, making the overall system more resilient to network failures or disruptions.

Enhanced security: With edge computing, sensitive data can be processed locally, reducing the risk and impact of data breaches and improving data privacy.

Lower bandwidth requirements: Edge computing reduces the amount of data that needs to be transmitted over the network, since only important data are sent to the cloud or data center. This reduces bandwidth requirements and associated costs.

However, there are also some challenges that need to be addressed, including:

Resource Constraints: Edge nodes typically have limited computing and storage resources compared to centralized data centers, which can limit the types and scale of applications that can be deployed on edge nodes.

Security: MEC introduces new security challenges, such as securing distributed computing resources and protecting sensitive data at the edge of the network. More specifically, an exploit can enable a rogue (or compromised) application to have access to non-authorized parts of RAM, thus allowing to steal or poison the data of the other applications running on the same physical host (side-channel attack or compromising). In a cloud environment, applications with higher security requirements would run on dedicated hardware. However, this is no longer possible in MEC because of the scarcity of resources.

Orchestration: MEC requires the management and orchestration of resources across multiple edge nodes and clusters, while

meeting the requirements of service chains, including security and end-to-end latency constraints.

In this paper, we focus on the orchestration challenges in MEC. These challenges are critical for the design of the MetaOS of the IoT-Edge-Cloud continuum. In particular, the paper brings the following contributions:

- Description and modelling of an optimization problem, called SLSCP (Security and Latency Service Chaining Problem) ;
- Algorithms that provide solutions for SLSCP ;
- Set-up of a platform that emulates a realistic Edge Computing environment ;
- Integration of the algorithms with the Kubernetes orchestrators of each cluster, demonstrating the feasibility of multi-clusters orchestration, with service-specific requirements.

In the following, we use interchangeably the terms 'VNF', 'function' or 'microservice'.

The problem of placing chains of services in the network with latency constraints has been extensively studied [11], [14], [10], [5], [4], [7]. However, not all have specifically considered the architecture of MEC ([12], [1]). Some works specifically deal with other constraints or objectives, such as energy [3] or security [6]. However, none has tackled yet service-specific requirements like latency and security jointly, and implemented the resulting algorithms on a production-ready environment for validation.

Section 2 gives an overview of the MEC architecture and the emulation platform. Section 3 details the security isolation model. Section 4 gives a formal description of SLSCP, ways to solve it, and the results of numerical experiments.

2 THE MEC EMULATION PLATFORM

The MEC architecture we consider in this work is based on the edge infrastructure described in [2], where small Data Centers (DC) are distributed down to the Cell Sites, as summarized in Fig. 1.

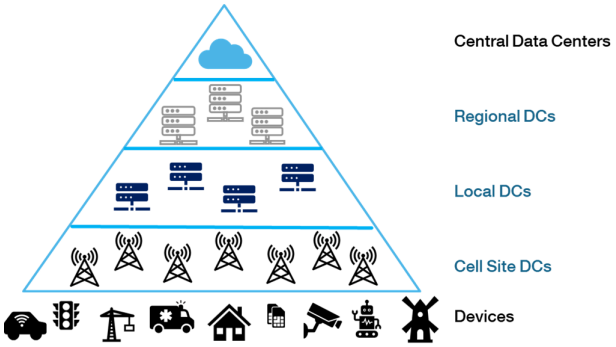

Figure 1: Proposed Edge Infrastructure Model

To demonstrate the feasibility in practice of our approach, we have setup an emulation platform that mimic the MEC architecture of Fig. 1. Each DC is materialized by a Kubernetes cluster in our platform.

For practical reasons, the whole platform infrastructure is virtualized. It is built upon best practice tools in industry. Firstly, it is based on LXC[1]. Then, the infrastructure building phase is automated thanks to Terraform[2] and Ansible[3]. Fig. 2 illustrates the internal structure of a cluster. HAProxy and keepalived keep the high access availability for flows, whether administration, customer, or intercluster flows. All clusters have their own private network and they share a common global DNS. Its role is to forward all external DNS requests to the right local DNS. Then, each local DNS synchronizes all local applications with the external-DNS applications.

We have installed Istio[4] to manage the global service mesh (multi-clusters) by selecting the architecture 'multi-primary on different network'. This architecture proposes for each cluster its own control plane, so that the clusters are independent from each other. We have also installed Kiali[5] to benefit from a feature-rich monitoring framework (cf. Fig. 3 for an illustration). Finally, we use ArgoCD[6] for automated deployment of service chains in the platform (cf. Fig. 3).

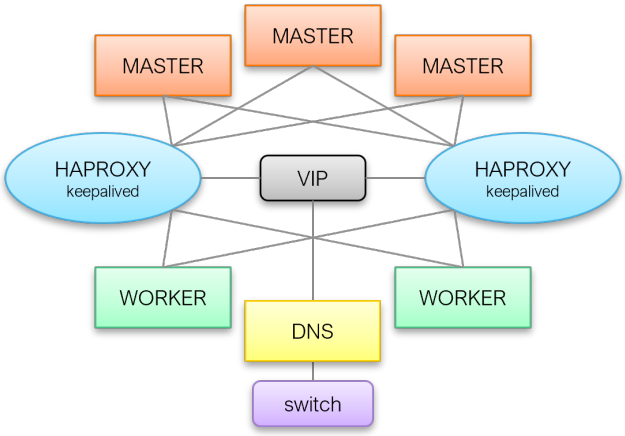

Figure 2: Structure of one cluster

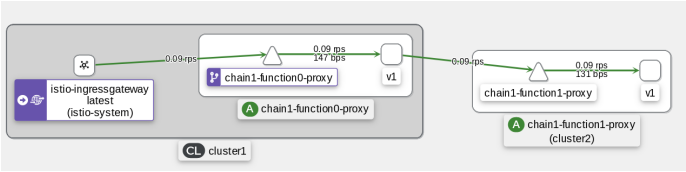

Figure 3: Kiali over Istio: monitoring of a Service Chain over 2 clusters

The platform is also used to perform measurements on the communication latency between the microservices of a Service Chain, thanks to Jaegger[7], a microservice tracing tool.

[1] https://linuxcontainers.org/lxc
[2] https://www.terraform.io
[3] https://www.ansible.com
[4] https://istio.io
[5] https://kiali.io
[6] https://argoproj.github.io/cd
[7] https://www.jaegertracing.io

Overall, the platform is composed of 3 clusters, each having 3 masters and 7 worker nodes (only 2 are shown in the figures for clarity purpose).

The integration of the placement algorithms with the Kubernetes orchestrators (each cluster has its own orchestrator) is done through the Kubernetes affinity mechanism. In Kubernetes, worker nodes can be given labels, chosen arbitrarily by the administrator. When microservices are deployed, it is possible to specify affinity rules with the node labels. The principle of our approach is to give each node a unique label. When the placement of microservices are determined (cf. the algorithm description in Section 4), we give affinity rules to the orchestrators of the different clusters, so that they comply with the placement we have computed. In order to keep the benefit of automatic scaling provided by Kubernetes, for each microservice, we specify affinity with several worker nodes, when possible.

3 SECURITY ISOLATION MODEL

Security requirements are a part of the Service Level Agreement (SLA) between the provider of IaaS (Infrastructure as a Service) and a Service Provider. The edge Data Centers must be secured by providing isolation of resources and data, because they belong to different tenants or network slices. Indeed, there are a number of attacks that can be performed by a rogue (or compromised) slice to steal or poison data used by another slice on the same physical host (cf. [12]). However, it is not possible to provide physical isolation to all slices, due to cost-efficiency and technical issues. Instead, we propose to physically isolate sensitive slices from less secure ones.

To this end, each function of a Service Chain defines an Isolation level as part of its requirements. This can be defined in the contracted SLA. If data processed by this micro-service are very sensitive, it will require a higher level of isolation. The Isolation level is freely chosen by the Service Chain owner, but of course the price charged by the IaaS provider will be impacted by this choice. The Isolation level can be typically set on a scale from 5 to 1, level 5 for National defense applications, level 4 for industrial ones, down to 1 for applications that don't process data or that the general public can use.

In addition, each function of a Service Chain is given a Trust level. This can be determined with a security assessment process, such as an audit, on the following criteria, for instance:

- Container image vulnerability scanning or audit
- Service development process (programming language, coding practice, libraries used, ...)
- Service Chain testing process

The Trust level is determined by the infrastructure provider. If the Service owner does not want to submit its service to the security audit, then it is given the lowest possible Trust level. Similarly to the Isolation level, the Trust level can be typically set on a scale from 1 to 5. Note that the model described in the Section 4 works with any kind of scale.

The Service Chain isolation can then be applied, by making sure that two micro-services are physically co-located if, and only if, the Trust level of one micro-service is higher or equal than the Isolation level of the other one, and reciprocally. This way, it is ensured that sensitive functions are only co-located with secure ones.

More formally, we denote $\langle T, I \rangle$ a micro-service with Trust level T and Isolation requirement I. We can define the following binary relation on the set of functions: $\langle T_1, I_1 \rangle \sim \langle T_2, I_2 \rangle \iff T_1 \geq I_2$ and $T_2 \geq I_1$. In other words, two functions are in relation if they can share a host. It is easy to check that this binary relation is reflexive and symmetric. However, it not transitive, (as a counter-example $\langle 3, 3 \rangle \sim \langle 5, 1 \rangle$ and $\langle 5, 1 \rangle \sim \langle 2, 2 \rangle$ but $\langle 3, 3 \rangle$ is not in relation with $\langle 2, 2 \rangle$). This non-transitivity is one of the reasons why the placement problem described in the next section is very hard. Indeed, the transitivity property would enable defining equivalence classes, and thus grouping functions by their class.

This approach has several advantages over physically isolated slices: first and foremost, it enables cost-efficient pseudo-isolation. Realistically, it is simply not feasible to have a physically isolated infrastructure for each sensitive service. Another advantage is that this kind of security isolation might not be needed all the time, it can be deployed when needed, thus saving resources and costs at the other times. For instance, isolation can be setup when a maintenance operation is in progress, i.e., when the risks of breach are higher, then it is released when the operation is finished.

4 SERVICE CHAINS PLACEMENT

When a Service Chain must be deployed, the administrator of the platform provides the placement algorithm with the description of the microservices that compose the chain. The algorithm also probes the platform in order to retrieve all needed information about the worker nodes and already running services. Then the algorithm is executed, and it gives as output the placement of microservices that meet all the service and security isolation requirements.

4.1 Problem description

In the so-called Security and Latency-aware Service Chaining Problem (SLSCP), we consider a set of nodes V, a set of microservices types F, and a set of Service Chains S. A given chain $k \in S$ is composed of an ordered set of functions (or microservices), denoted F_k. A given function $f \in F$ is characterized by a Trust T_f and Isolation level I_f (cf. Section 3), and by requirements in terms of RAM, CPU, and storage, denoted respectively R_f^{ram}, R_f^{cpu}, R_f^{sto}. The aim is to attribute a placement for each function onto a node.

Each node $u \in V$ is characterized by a cost for deploying a function on this node c_u, and its capacities in terms of RAM, CPU and storage, denoted, respectively, C_u^{ram}, C_u^{cpu}, C_u^{sto}.

Each Service Chain $k \in S$ has a requirement for the maximum allowed end-to-end communication latency, noted L_k. We assume the communication latency of using a link between two nodes u and $v \in V$ is constant and is noted L_{uv}. We make use of the emulation platform to perform experimental measurements of the latency of all links. The latency of the whole chain is the sum of all the links used by consecutive functions in the chain.

Some functions are marked as backup for another one, so they cannot be both on the same node (anti-affinity constraints).

Since microservices are placed on physical hosts in an Edge Data Center, we assume that all nodes are connected to all others, with an over-provisioned bandwidth. In consequence, there is no routing/bandwidth issues in SLSCP.

Table 1: Notations summary

Sets	
V	Set of nodes (index u)
F	Set of functions (index f)
S	Set of chains (index k)
F_k	Set of functions in chain $k \in S$
Constants	
I_f	Isolation level of function f
T_f	Trust level of function f
R_f^{cpu}	Requirement in vCPU of function f
C_u^{cpu}	Capacity in vCPU of node u
R_f^{ram}	Requirement in RAM of function f
C_u^{ram}	Capacity in RAM of node u
R_f^{stor}	Requirement in storage of function f
C_u^{stor}	Capacity in storage of node u
c_u	Cost of using node u
l_{uv}	Latency induced by using link (u, v)
L_k	Maximum end-to-end latency admitted by chain k
S_{fg}	Indicator whether f and g are consecutive
B_{fg}	Indicator whether g is a backup for f
Variables	
x_{uf}	Binary indicator of function f placed on node u
y_u	Binary indicator of node u in use

4.2 Formulation

The decision variables are:

$x_{uf} \in \{0, 1\}$, that takes the value 1 if function f is placed on node u, 0 otherwise, $\forall u \in V, \forall f \in F$.

$y_u \in \{0, 1\}$, that takes the value 1 if node u is used, 0 otherwise, $\forall u \in V$.

Table 1 sums up the notation, and a compact formulation is given (cf. Equations 1-8).

$$\min \sum_{k \in S} \sum_{u,v \in V} \sum_{f,g \in S_k} x_{uf} x_{vg} S_{fg} l_{uv} \tag{1}$$

subject to

$$\sum_{f \in F} R_f^{cpu} x_{uf} \leq C_u^{cpu} y_u \qquad \forall u \in V \tag{2}$$

$$\sum_{f \in F} R_f^{ram} x_{uf} \leq C_u^{ram} y_u \qquad \forall u \in V \tag{3}$$

$$\sum_{f \in F} R_f^{stor} x_{uf} \leq C_u^{stor} y_u \qquad \forall u \in V \tag{4}$$

$$I_f (x_{uf} + x_{uf'} - 1) \leq T_{f'} x_{uf'} \qquad \forall u \in V, \forall f, f' \in F \tag{5}$$

$$\sum_{u \in V} x_{uf} = 1 \qquad \forall f \in F \tag{6}$$

$$\sum_{u,v \in V} \sum_{f,g \in S_k} x_{uf} x_{vg} S_{fg} l_{uv} \leq L_k \qquad \forall k \in S \tag{7}$$

$$x_{uf} + x_{ug} \leq 2 - B_{fg} \qquad \forall u \in V, \forall f, g \in F \tag{8}$$

In this formulation, the objective (Eq. 1) is to minimize the overall communication latency of all chains. Eq. 2-4 are the capacity constraints. Eq. 5 enforce the security isolation constraints. Eq. 6 ensure that each microservice is placed once, while Eq. 7 enforce the maximum latency for each service chain. Finally, Eq. 8 make sure that backup functions are not located on the same node as the function they protect.

In order to make the formulation linear, we define a new set $F^C \subset F \times F$ as follows: $\forall f, g \in F, (f, g) \in F^C \Leftrightarrow S_{fg} = 1$. In other terms, F^C is the set of couples of consecutive functions. We define a new variable as follows: $z_{uvfg} \in \{0, 1\}$, which takes the value 1 if $x_{uf} = x_{vg} = 1$, and the value 0 otherwise, $\forall u, v \in V, \forall (f, g) \in F^C$. This variable is defined only for functions that are consecutive in a chain. Equation 7 becomes:

$$\sum_{u,v \in V} \sum_{(f,g) \in F^C} z_{uvfg} l_{uv} \leq L_k \qquad \forall k \in S \tag{9}$$

$$x_{uf} + x_{vg} - 1 \leq z_{uvfg} \qquad \forall u, v \in V, \forall (f, g) \in F^C \tag{10}$$

$$z_{uvfg} \leq x_{uf} \qquad \forall u, v \in V, \forall (f, g) \in F^C \tag{11}$$

$$z_{uvfg} \leq x_{vg} \qquad \forall u, v \in V, \forall (f, g) \in F^C \tag{12}$$

4.3 Multi-criteria Formulations

With respect to energy savings, it is valuable to have the service chains span over the least workers as possible. In this way, when the load is low, it becomes possible to turn off unneeded workers. Therefore, in order to have the functions packed as much as possible, we introduce a cost c_u for using node u (i.e., at least one function is placed on it). This cost can be interpreted as a power consumption cost. We then define the SLSCP_cost problem as:

$$\min \sum_{u \in V} c_u y_u \tag{13}$$

$$\text{with constraints } (2) - (8) \tag{14}$$

We can then define a new problem, denoted SLSCP_2steps_cost, which consists in a two step process: first we find a solution to SLSCP_cost, which yields a minimum cost denoted c^*, then we solve SLSCP with the additional following constraint:

$$\sum_{u \in V} c_u y_u \leq c^* \tag{15}$$

It is also possible to perform the two-steps process by first minimizing the overall latency, then adding a constraint on the overall latency and minimizing the cost. This approach is denoted SLSCP_2steps_lat in the following.

4.4 Greedy heuristics

For comparison purpose, we also have designed a greedy heuristics, in which we perform the following steps:

- Sort the service chains, by increasing maximum allowed latency, then by decreasing number of functions
- Sort the hosts by cluster id, then by increasing trust level, then by decreasing available CPU resource.

These sorts are designed so that the chains with the highest requirements are placed first, to prevent placement failure as much as possible. Then we take each chain one by one in order, and place its functions on the first host possible, checking each host in order.

Hosts are re-sorted after placing a function. If a function cannot be placed, or if a chain violates its isolation or its maximum allowed latency constraint, it means that greedy has failed to find a feasible solution for this instance.

We denote *greedy_rand*, a variant in which the service chains are not sorted, but are randomly shuffled instead.

4.5 Lower Bounds

We can obtain a very good lower bound on the cost (denoted c^{LB}) by relaxing the latency constraints in the SLSCP_cost problem. More formally, we denote SLCP_cost_LB the problem with objective (13), under constraints (2) to (6) and (8). This problem is solved much quicker than SLSCP_cost, we could check that for all instances used in the numerical results (Section 5), it is solved in less than 1 second.

For the lower bound on latency (denoted L^{LB}), we propose the following heuristics: we go through all the functions in a chain, if two consecutive functions cannot be placed on the same host, we add the inter-node latency to the latency lower bound. At the end of the chain, we get a lower bound that is the minimum latency for this chain when we relax the capacity constraints. This lower bound is computed very efficiently, in linear time vs. the number of functions.

5 NUMERICAL EXPERIMENTS

We have implemented the algorithms described in Section 4 in Python [13] with Pyomo [8] and the Cplex [9] solver. The experiments were run on a Linux machine with a quad-core Xeon Silver 4112 CPU at 2.60GHz and 192 GB of RAM. We have generated a large number of random instances, with varying number of functions to place. The functions are grouped in chains of random length from 2 to 6. The Isolation levels are selected randomly from 1 to 5 (with a lower weight on higher levels, then the Trust level is selected randomly between the Isolation level and 5 (a Service Chain provider cannot request an Isolation level higher than the Trust level of its function). The latency is chosen randomly, however we ensure that it is a feasible value, given the inter-node latency. The resource requirements are the same for all functions. The cost of using a host is set to 1, therefore the cost represents the number of hosts used.

We can observe on Fig. 4 and 5 that the optimal algorithm is always significantly better than our baseline, the greedy algorithm. For 25 functions, the improvement in cost is around 23%, and in latency around 21%. On all figures (except Fig. 7), greedy_rand and greedy are superimposed, therefore greedy_rand is omitted for figure clarity reason. The algorithm opt_lat (resp. opt_cost) is quite inefficient in terms of cost (resp. latency). However, this is completely mitigated by the usage of a 2-steps approach. Indeed, 2steps_cost, is still much better than greedy, in terms of latency.

Fig 6 shows the execution time for each algorithms, while Fig. 7 shows the proportion of failed instances for greedy and greedy_rand. As expected, greedy_rand is much worse because by placing chains with stricter requirements first (in greedy), we decrease the likelihood of failure. However, even for greedy, the proportion of failure is quite high (between 5% and 30% for most higher function counts). The exact algorithms do not fail because the instances were designed to have feasible solutions. Finally, Fig. 8 shows the maximum gap between greedy and the optimum. It is computed as $(c^{greedy} - c^{LB})/c^{LB}$ for the cost and $(L^{greedy} - L^{LB})/L^{LB}$ for latency. It is around 15% for the cost and around 20-25% for the latency, except with low counts of functions.

In conclusion, the exact resolution approaches described in this paper have a longer execution time, but they are much better than the baseline in terms of solution quality. In particular, they offer guarantees on the physical isolation of micro-services, as well as guarantees on the end-to-end communication delay of chains. These guarantees cannot be offered with a greedy heuristics approach, even though they are essential to an efficient orchestration algorithm.

6 CONCLUSION

In conclusion, in the work described in this paper, we have made the following achievements:

- Definition, modelling and solving of the SLSCP problem
- Integration with Kubernetes orchestrator, to provide multi-cluster and service-specific orchestration
- Description of an approach for Security-by-Orchestration
- Setup of an emulation platform, in accordance with industry best-practice.

In consequence, we could demonstrate the feasibility of a security-by-orchestration approach.

Currently, the platform only supports so-called bare-metal clusters. We are now in the process of integrating Amazon Web Services (AWS) clusters into it. Afterwards, we will also integrate a GCP (Google Cloud Platform) cluster, in order to make the platform a true hybrid, multi-clusters edge computing emulation platform.

We are also investigating ways to improve the performance of the placement algorithm, with respect to execution time.

ACKNOWLEDGMENTS

This work was supported by the European Union H2020 Project DEDICAT 6G under grant no. 101016499.

REFERENCES

[1] T. W. Nowak al. 2021. Verticals in 5G MEC-Use Cases and Security Challenges. *in IEEE Access* 9 (2021), 87251–87298. https://doi.org/10.1109/ACCESS.2021.3088374

[2] R. Artych, K. Bocianiak, Y. Carlinet, W. Niewolski, N. Perrot, A. Podlasek, and J. Wary. 2022. Security Constraints for Placement of Latency Sensitive 5G MEC Applications. In *2022 9th International Conference on Future Internet of Things and Cloud (FiCloud)*. IEEE Computer Society, Los Alamitos, CA, USA, 40–45. https://doi.org/10.1109/FiCloud57274.2022.00013

[3] H. Badri, T. Bahreini, D. Grosu, and K. Yang. 2020. Energy-Aware Application Placement in Mobile Edge Computing: A Stochastic Optimization Approach. *in IEEE Transactions on Parallel and Distributed Systems* 31, 4 (April 2020), 1. https://doi.org/10.1109/TPDS.2019.2950937

[4] S. Bi, L. Huang, and Y. A. Zhang. 2020. Joint Optimization of Service Caching Placement and Computation Offloading in Mobile Edge Computing Systems. *in IEEE Transactions on Wireless Communications* 19, 7 (July 2020), 4947–4963. https://doi.org/10.1109/TWC.2020.2988386

[5] B. Brik, P. A. Frangoudis, and A. Ksentini. 2020. Service-Oriented MEC Applications Placement in a Federated Edge Cloud Architecture. In *International Conference on Communications (ICC) pp*, Icc Ieee (Ed.). 1-6, 2020–2020.

[6] L. Caviglione and M. Gaggero. 2021. Multiobjective Placement for Secure and Dependable Smart Industrial Environments. *in IEEE Transactions on Industrial Informatics* 17, 2 (February 2021), 1298–1306. https://doi.org/10.1109/TII.2020.2978771

[7] L. Ferrucci et al. 2020. Latency preserving self-optimizing placement at the edge. In *Proceedings of the 1st Workshop on Flexible Resource and Application Management on the Edge.*

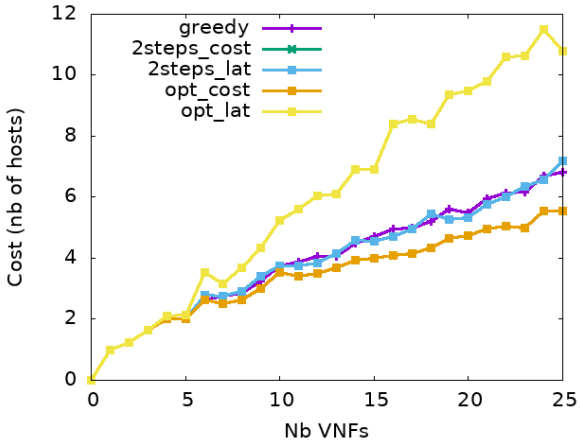

Figure 4: Cost results

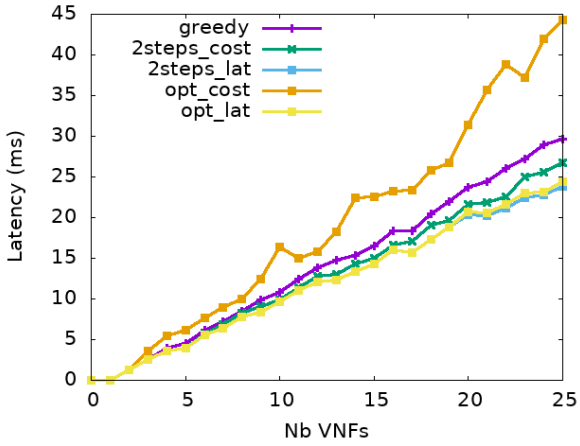

Figure 5: Latency results

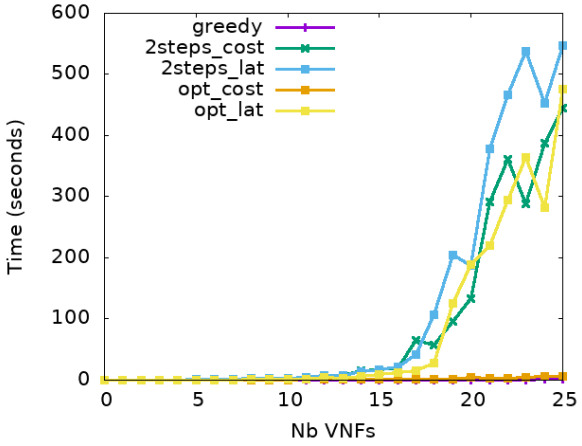

Figure 6: Computing Time

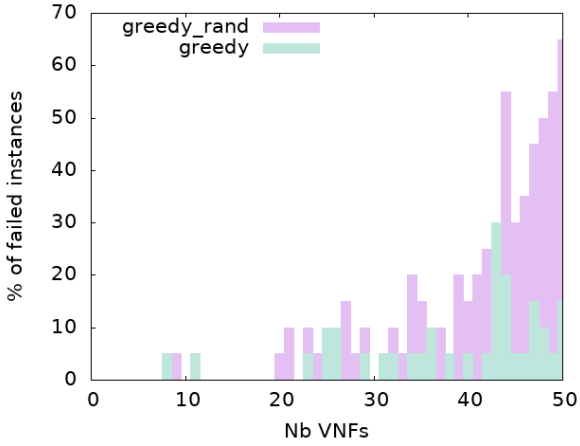

Figure 7: Greedy failed instances

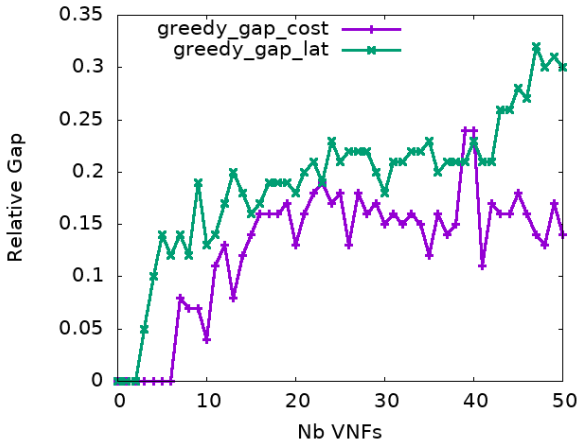

Figure 8: Greedy Gap

[8] William E. Hart, Carl D. Laird, Jean-Paul Watson, David L. Woodruff, Gabriel A. Hackebeil, Bethany L. Nicholson, and John D. Siirola. 2017. *Pyomo–optimization modeling in python* (second ed.). Vol. 67. Springer Science & Business Media.

[9] ILOG, Inc. 2006. ILOG CPLEX: High-performance software for mathematical programming and optimization. http://www.ilog.com/products/cplex/.

[10] R. Mahmud, A. N. Toosi, K. Ramamohanarao, and R. Buyya. 2020. Context-Aware Placement of Industry 4.0 Applications in Fog Computing Environments. *in IEEE Transactions on Industrial Informatics* 16, 11 (November 2020), 7004–7013. https://doi.org/10.1109/TII.2019.2952412

[11] Ahlam Mouaci, Éric Gourdin, Ivana Ljubić, and Nancy Perrot. 2023. Two extended formulations for the virtual network function placement and routing problem. *Networks* (03 2023). https://doi.org/10.1002/net.22144

[12] 5G PPP White Paper. 2021. Edge Computing for 5G Networks. *https://bscw.5g-ppp.eu/pub/bscw.cgi/d397473/EdgeComputingFor5GNetworks.pdf* (2021).

[13] Guido Rossum. 1995. Python Reference Manual, 1995. http://www.python.org.

[14] M. Wang, B. Cheng, W. Feng, and J. Chen. 2019. An Efficient Service Function Chain Placement Algorithm in a MEC-NFV Environment. *IEEE Global Communications Conference (GLOBECOM* 2019 (2019), 1–6. https://doi.org/10.1109/GLOBECOM38437.2019.9013235

Received 11 February 2024; accepted 12 March 2024; revised 15 March 2024